

Name of the Student	V Sunil Kumar
Reg. No	192425292
GitHub Link	https://github.com/192425292simats-byte/MLA0403-DEEP-LEARNING

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning
SLOT : A

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10%
Assessment Tool Weightage: 40%

Duration: 90 minutes
Mark : 25

Code of Conduct:

I (V Sunil Kumar / 192425292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: V Sunil

Criteria	Understanding of Industry Problem (2)	Application of Engineering Concepts (2.5)	Solution Design (2.5)	Use of Tools (2)	Analysis (1)	Total (10)
Weightage	20 %	25%	25%	20 %	10 %	100%
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Q1. Transfer Learning – Skin Cancer Detection

Answer:

A healthcare startup wants to develop an AI system for detecting skin cancer from dermoscopic images. Since collecting and labeling a large medical image dataset is difficult, **Transfer Learning** can be used with a pre-trained CNN.

1. Understanding of the Industry Problem

Skin cancer detection requires accurate classification of dermoscopic images into classes such as **benign and malignant**. The major challenge is the limited availability of labeled medical images. Training a CNN completely from scratch may cause overfitting and require a large dataset.

2. Proposed Solution

A pre-trained **ResNet50** model trained on ImageNet can be selected. ResNet50 is suitable because it has strong feature extraction capability and uses **residual connections**, which allow deeper networks to learn effectively.

The architecture can be designed as:

Dermoscopic Image → Preprocessing → ResNet50 Feature Extractor → Global Average Pooling → Dense Layer → Softmax/Sigmoid → Cancer Prediction

3. Frozen and Fine-Tuned Layers

Initially, most of the ResNet50 convolutional layers are **frozen** because they already contain useful features such as edges, textures, shapes, and patterns.

- Early convolutional layers → **Frozen**
- Middle layers → **Initially frozen**
- Final convolutional block → **Fine-tuned**
- Original ImageNet classification layer → **Removed**
- New classification layer → **Added and trained**

For example, the final layer can use a sigmoid activation for binary classification:

Output = Benign / Malignant

After initial training, some deeper ResNet layers can be unfrozen and fine-tuned using a **small learning rate**.

4. Tools and Technologies

- Python
- TensorFlow/Keras
- ResNet50

- OpenCV
- NumPy
- Dermoscopic image datasets such as ISIC

Data augmentation techniques such as rotation, flipping, cropping and brightness adjustment can increase the effective training data.

5. Analysis and Performance

Transfer learning reduces training time because the model does not learn all visual features from the beginning. Freezing early layers reduces the number of trainable parameters and helps prevent overfitting.

Performance can be evaluated using:

- Accuracy
- Precision
- Recall/Sensitivity
- F1-score
- ROC-AUC
- Confusion Matrix

For healthcare applications, **recall/sensitivity is particularly important**, because missing a malignant case can have serious consequences.

Conclusion

The proposed ResNet50 transfer learning system can achieve better performance with limited labeled data than training a CNN from scratch. Fine-tuning the deeper layers allows the model to adapt general image features to specific skin-lesion characteristics.

Q2. DenseNet and PixelNet – Road Scene Analysis

Answer:

A smart city requires an automated road-scene analysis system that identifies **roads, pedestrians, vehicles and traffic signs at pixel level**. This is a semantic segmentation problem.

1. Understanding of the Industry Problem

For autonomous transportation and smart-city applications, simply detecting an object is not sufficient. The system must determine the class of **each pixel** in the image. Accurate segmentation is required for navigation, traffic monitoring and road safety.

2. Proposed Architecture

A combination of **DenseNet and PixelNet** can be designed.

Input Road Image → DenseNet Backbone → Feature Extraction → PixelNet Pixel-Level Processing → Upsampling → Segmentation Map

DenseNet can act as the feature extractor, while PixelNet can perform pixel-level prediction.

3. Role of DenseNet

DenseNet connects each layer to subsequent layers through dense connections.

Instead of:

Layer 1 → Layer 2 → Layer 3

DenseNet uses:

Layer 1 → Layer 2, Layer 3, Layer 4

and earlier features are reused by later layers.

This provides:

- Better feature reuse
- Improved gradient flow
- Reduced redundant feature learning
- Strong representation of edges and textures

DenseNet is therefore useful for extracting features such as vehicles, pedestrians, road boundaries and traffic signs.

4. Role of PixelNet

PixelNet performs **pixel-level classification**. Each pixel is assigned to a particular class:

- Road
- Pedestrian
- Vehicle
- Traffic sign
- Background

The final output is a segmentation map where different pixel regions represent different objects.

5. Computational Efficiency

Dense connections allow the network to reuse previously learned features rather than learning the same information repeatedly. This can improve parameter efficiency.

Pixel-level processing provides detailed segmentation, which is important for small objects such as traffic signs and pedestrians.

6. Tools

- Python
- TensorFlow/Keras or PyTorch
- DenseNet
- PixelNet-style segmentation architecture
- OpenCV
- Cityscapes or similar road-scene datasets
- GPU/CUDA for training

7. Analysis and Performance

The system can be evaluated using:

- Pixel Accuracy
- Mean Intersection over Union (mIoU)
- Precision
- Recall
- F1-score
- Inference time

mIoU is particularly useful because it measures how accurately the predicted segmentation overlaps with the actual segmentation.

Conclusion

DenseNet provides strong and reusable hierarchical features, while PixelNet performs detailed pixel-level classification. Their integration provides accurate road-scene segmentation while improving feature reuse and computational efficiency.

Q3. Bidirectional RNN and Encoder–Decoder – Multilingual Chatbot

Answer:

A multinational customer-service company needs a multilingual chatbot that can translate customer queries while preserving their contextual meaning. An **Encoder–Decoder sequence-to-sequence architecture with Bidirectional RNNs** can solve this problem.

1. Understanding of the Industry Problem

Customer queries can contain long sentences and contextual information. A translation system must understand the complete input sentence before generating the translated output.

For example:

Input:

“Where can I change my booking?”

The chatbot must understand the relationship between words before producing the translation.

2. Proposed Architecture

The architecture can be represented as:

Input Sentence → **Word Embedding** → **Bidirectional RNN Encoder** → **Context Representation** → **Decoder RNN** → **Output Words**

The encoder reads the input sequence in both directions.

Forward:

Word₁ → Word₂ → Word₃ → Word₄

Backward:

Word₄ → Word₃ → Word₂ → Word₁

The outputs from both directions are combined to create a richer representation.

3. Encoder

The **Bidirectional RNN encoder** processes the input from both left-to-right and right-to-left.

For example:

“The customer cancelled the order yesterday.”

The meaning of a word can depend on both previous and following words. Bidirectional processing allows the encoder to consider information from both directions.

4. Decoder

The decoder generates the translated sentence one word at a time.

Context Vector → **Decoder** → **Word₁** → **Word₂** → **Word₃** → ...

An attention mechanism can also be included so that the decoder focuses on the most relevant encoder states when generating each word.

5. Advantages of Bidirectional Processing

Bidirectional RNNs:

- Capture past and future context
- Improve understanding of sentence structure
- Handle contextual word meanings better
- Improve translation of longer sentences
- Provide richer encoder representations

6. Tools

- Python
- TensorFlow/Keras or PyTorch
- NLP preprocessing libraries
- Tokenization
- Word embeddings
- BLEU score for translation evaluation
- Multilingual datasets

7. Analysis

The model can be evaluated using:

- BLEU score
- Translation accuracy
- Perplexity
- Validation loss
- Human evaluation

The model can be compared with a normal unidirectional RNN. A higher BLEU score and lower validation loss indicate improved translation quality.

Conclusion

The Bidirectional RNN encoder captures information from both directions, while the decoder generates the target-language sequence. Adding attention further improves the ability to preserve contextual meaning, making the architecture suitable for multilingual customer-service translation.

Q4. BPTT and LSTM – Stock Price Prediction

Answer:

A financial analytics company wants to forecast stock prices using historical market information. Since stock prices form a time sequence, an **LSTM-based recurrent neural network** is appropriate.

1. Understanding of the Industry Problem

Stock prices depend on historical patterns and time-dependent information. Important information may occur many time steps before the current prediction. A traditional RNN may have difficulty remembering such long-term dependencies because of the **vanishing gradient problem**.

2. Proposed LSTM Architecture

The system can be designed as:

Historical Stock Data → Preprocessing → LSTM Layer → Dense Layer → Predicted Stock Price

Input features may include:

- Open price
- High price
- Low price
- Closing price
- Trading volume

A sequence of previous days can be used to predict the next day's price.

3. LSTM Structure

An LSTM contains:

- Forget gate
- Input gate
- Cell state
- Output gate

The **forget gate** determines which old information should be removed.

The **input gate** determines which new information should be stored.

The **cell state** carries important information through time.

The **output gate** determines the information used for the current prediction.

4. Backpropagation Through Time

BPTT is used to train recurrent networks.

The process is:

Input Sequence → LSTM → Prediction → Loss Calculation → Backward Through Time → Weight Update

The model calculates the prediction error and propagates the error backward through the time steps.

For example:

Day 1 → Day 2 → Day 3 → Day 4 → Prediction

The error at the prediction is propagated backward through Day 4, Day 3, Day 2 and Day 1.

The optimizer then updates the model weights to reduce the prediction error.

5. Why LSTM Instead of Traditional RNN?

Traditional RNN	LSTM
Difficult to learn long-term dependencies	Better at long-term dependencies
More affected by vanishing gradients	Gates help control information flow
Simple memory mechanism	Cell state provides persistent memory
Less suitable for long financial sequences	More suitable for sequential financial data

6. Tools

- Python
- TensorFlow/Keras
- Pandas
- NumPy
- Matplotlib
- Historical stock dataset
- GPU for faster training

7. Performance Analysis

The model can be evaluated using:

- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Error (MAE)

Actual and predicted prices can also be plotted on a graph.

A lower RMSE/MAE indicates better prediction performance.

Conclusion

LSTM is more suitable than a traditional RNN because its gated architecture can retain useful information for longer periods. BPTT trains the LSTM by propagating errors across the sequence, allowing it to learn temporal patterns in historical market data.

Note: Stock-price prediction is inherently uncertain, so the model should be treated as a forecasting aid rather than a guarantee of future prices.

Q5. Integrated Deep Learning Solution – Automatic Video Captioning

Answer:

A media streaming company wants to automatically generate captions for uploaded videos. An integrated architecture combining **Transfer Learning for visual feature extraction** and an **LSTM Encoder–Decoder for caption generation** can be developed.

1. Understanding of the Industry Problem

Videos contain a large amount of visual information. The system must first understand what is happening in the video and then convert that information into natural-language captions.

For example:

Video: A person is playing football.

Generated

Caption:

“A person is playing football.”

2. Proposed Architecture

The complete system can be represented as:

Video → Frame Extraction → Pre-trained CNN → Visual Features → LSTM Encoder → Context Representation → LSTM Decoder → Caption

3. Transfer Learning Component

A pre-trained CNN such as **ResNet50** or **EfficientNet** can be used.

The CNN is initially trained on a large image dataset and already understands general visual features.

For each video:

1. Extract frames at regular intervals.
2. Resize and normalize the frames.
3. Pass each frame through the pre-trained CNN.
4. Extract feature vectors.
5. Provide the sequence of visual features to the LSTM.

The final classification layer of the CNN is removed because the objective is to obtain **visual feature representations**, not image-classification labels.

4. LSTM Encoder

The LSTM encoder receives the sequence of visual features.

Frame	1	→	Feature	1
Frame	2	→	Feature	2

Frame 4 → Feature 4

The LSTM learns the temporal relationship between these frames and creates a contextual representation of the video.

5. LSTM Decoder

The decoder converts the learned visual representation into words.

For example:

<START> → A → man → is → playing → football → <END>

During training, the model learns the relationship between visual features and corresponding captions.

An **attention mechanism** can be added so that the decoder focuses on relevant video frames while generating each word.

6. Role of Each Component

Component	Role
Video Frames	Provide visual information
Pre-trained CNN	Extracts visual features
Transfer Learning	Reduces training requirements
LSTM Encoder	Learns temporal relationships
LSTM Decoder	Generates caption words
Attention	Focuses on relevant frames/features
Tokenizer	Converts words into numerical representations

7. Tools

- Python
- TensorFlow/Keras or PyTorch
- OpenCV
- Pre-trained ResNet50/EfficientNet
- LSTM
- NLP tokenizer
- GPU/CUDA
- Video-caption datasets such as MSR-VTT or MSVD

8. Performance Analysis

The generated captions can be evaluated using:

- BLEU
- ROUGE

- METEOR
- CIDEr
- Human evaluation

The system should also be tested on different video categories, lighting conditions, camera movements and sentence lengths.

9. Advantages of the Integrated Solution

The combined system provides:

- Effective visual feature extraction
- Reduced training time through transfer learning
- Temporal understanding through LSTM
- Natural-language caption generation
- Better handling of sequential video information
- Improved accessibility for users with hearing difficulties

Conclusion

Transfer Learning provides powerful visual representations without training a CNN from scratch. The LSTM Encoder understands the sequence of visual events, while the LSTM Decoder converts these features into meaningful captions. Adding attention improves the connection between specific video frames and generated words. Therefore, the integrated architecture provides a practical solution for automatic video caption generation.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	20%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis